REVIO STOCKSENSE

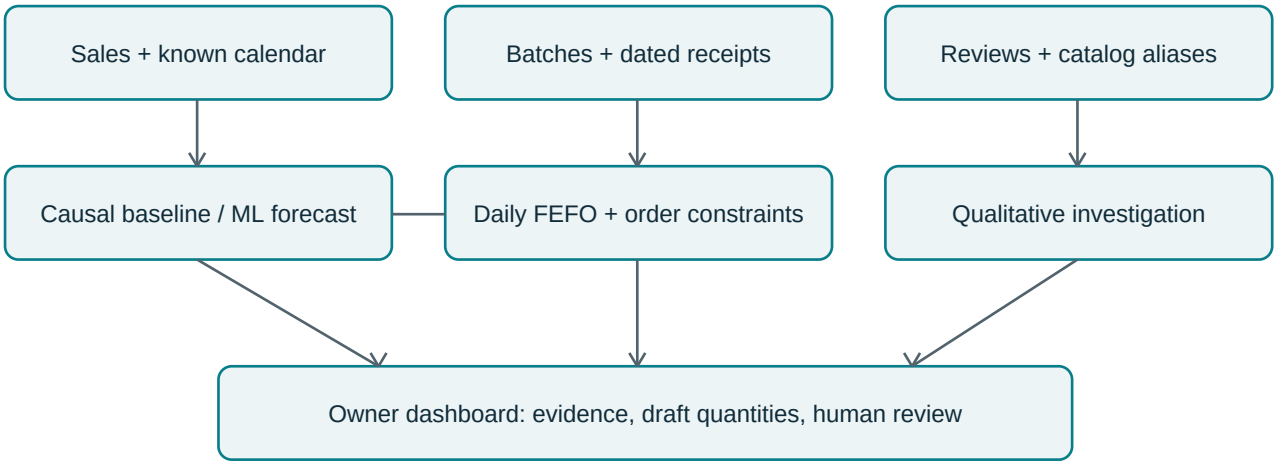
Complete Project Explanation

An accessible walkthrough with worked examples, engineering decisions and40 interview questions.

17 October 2026
Independent proposal for Cynbit AI/ML Engineer Internship

Research claims, simulated measurements and future proposals are distinguished throughout. No live Revio integration or real-store ROI is claimed.

System at a glance



A learning companion covering architecture, model reasoning, inventory examples, decisions and interview practice. All results refer to synthetic demonstration data.

Part I - End-to-end walkthrough

Start with the business decision

A kirana owner wants to know whether to buy more of a product. A forecast answers how much demand may occur. A separate inventory policy asks whether existing stock and supplier receipts can satisfy it without expiry or excess. Customer reviews add reasons to investigate a product; they do not measure units sold.

This separation makes errors easier to understand. If demand is wrong, inspect training and recent observations. If a quantity is wrong despite a sensible forecast, inspect batch dates, receipt timing and policy constraints. No component places an order.

Repository map

Path	Purpose
app/app.py	Seven-section Streamlit UI, session state and cached computation
src/data.py	Six-table validation, CSV handling, demand masking, summaries, SQLite
src/forecasting.py	Origin-only features, training, prediction, metrics, chronological evaluation
src/inventory.py	Periodic policy, constrained rounding, daily FEFO, explanations
src/seasonal_model.py	Per-product joint seasonal, trend, weekday and festival count model
src/calendar_data.py	Public calendar snapshot normalization and provenance
src/seasonality.py	Monthly profile and comparable event windows
src/reviews.py	Product alias and English intent rules
scripts/generate_data.py	Seeded synthetic ledger and snapshot generator
scripts/evaluate.py	Reproduce model comparisons and prediction CSV
scripts/build_reports.py	Export editable report sources to PDFs
data/sample	Six CSV tables and generation metadata
evaluation	Measured metrics and backtest product-day predictions
tests	Data, business-rule, causality and UI integration checks
docs	Research, decisions, interview guide, recovery and development logs
reports	Three editable reports and rendered PDFs

From CSV bytes to a valid snapshot

`read_csv` accepts UTF-8 data within a10MB limit. `validate` copies each table, normalizes text, checks headers and converts numeric/date fields. It rejects nonfinite or negative quantities, malformed dates, duplicate keys, invalid references and impossible recorded stock balances. All tables must pass before the UI replaces its session snapshot. A failed form leaves the previous data active.

Dates become pandas timestamps. Blank batch dates become NaT, preserving unknown status. The latest sales date is the snapshot's as-of date: the bundled demonstration remains reproducible instead of drifting with the computer clock. SQLite roundtrip serializes dates back to ISO strings and validates them again on load. Writes use a temporary database and atomic replacement.

Observed sales versus demand

Suppose a shelf begins with three bottles and sells all three. Actual demand could be three or fifteen. The program does not know. It masks a known stockout or exhausted availability instead of training on three as if demand were fully observed. Missing dates are also unknown. A true zero with available stock stays zero.

This is a pragmatic baseline, not a complete censored-demand estimator. Discarding constrained days may select unusually quiet days and underestimate demand. The generator records latent demand so evaluation can illustrate that issue, but production data cannot supply it directly.

Legacy boosting comparator training rows

Take an origin at the end of18 June2026 and a target seven days later. History features use observations no later than18 June. The target weekday/month and event proximity are known calendar features for25 June. The target is observed uncensored sales on25 June. A training cutoff earlier than25 June would forbid this row.

`features` creates30 rows per eligible origin, one per future horizon. `train` samples origins weekly to keep the demonstration lightweight. It stacks all SKUs into a global dataset; product identity is categorical. The model learns nonlinear splits such as different calendar behavior for different products. The ensemble improves successive residual errors; it does not memorize a fixed summer uplift rule.

No recursive prediction loop feeds yesterday's prediction into today's lag. This is direct multi-horizon regression: every horizon uses the same known origin history plus that horizon's calendar features. The tradeoff is more training rows and an explicit maximum30-day planning horizon.

Why there are four predictors

A model is useful only if it improves a credible simple approach. A28-day average and weekly seasonal naive forecast are cheap, understandable benchmarks. Four validation windows select the lowest pooled MAE. Twelve later 30-day rolling test windows span 360 days. Seasonal Poisson regression wins selection and jointly fits each product's smooth annual curve, weekdays, trend and a pooled dated festival effect. It is refitted on available history for the next30 days. With less than a full annual span it falls back to the average; without adequate recent history it withholds predictions.

For uploaded data, demo results are irrelevant. The UI starts with the moving average. Running that snapshot's chronological evaluation selects a method for that snapshot. Very short histories cannot support the full evaluation protocol. Point forecasts have data-support labels, not claimed confidence intervals.

How to read the measured results

The selected seasonal model has rolling test MAE2.860, RMSE3.921, WAPE18.520% and bias-0.040. MAE is average absolute daily error in units; WAPE is aggregate numerical error, not classification accuracy.

The same-data boosting MAE is2.964 and average MAE3.374: relative improvements of3.5% and15.2%. There are4,200 observed product-days across360 test days. All four broad seasons improve, but not every SKU. The no-history SKU is excluded. Latent simulation diagnostics are separate. Old v1 metrics use different dates and a shorter protocol, so cannot be compared directly.

Walk through the inventory engine

`periodic_quantity` implements the basic policy: forecast demand plus buffer minus usable stock and incoming supply plus backorders, floored at zero. With20 units/day, lead4, review7, safety30, stock80 and incoming20, target250 minus position100 gives150.

`simulate` goes day by day. Confirmed supply arrives on its expected date. Batches already expired are excluded. Demand consumes the earliest eligible expiry first. Unmet daily demand is recorded as shortage rather than automatically carried as an order. Existing backorders are a separate first-day obligation. This simplifies lost-sales behavior and should be reviewed for a real store.

`plan` subtracts projected expiry loss from effective supply during protection. It flags shortages before the new order's arrival. It caps new units by demand still unsatisfied during the new receipt's assumed life and by unit storage capacity at arrival. It then honors pack and MOQ constraints. A cap20 with pack12 can allow12, not24; if MOQ24, it withholds the order.

Unknown expiry blocks automatic ordering until stock is verified. Lead plus review greater than30 days withholds quantity because there is no supported forecast for the full period. A zero draft can therefore mean sufficient stock, a binding constraint or insufficient evidence; read the status and explanation.

Batch expiry example

Eighty yogurt units expire after five future sellable days. Demand6/day consumes30; fifty remain at risk. With two batches, FEFO gives the earlier batch first claim on those30 units; the later batch cannot also claim them. Recorded expiry dates remain unchanged. Blank risk beyond30 days means unknown, not safe. Expired items are never recommended for sale.

Seasons, events and reviews

The monthly chart shows historical observed means, which can reflect seasonality and trend together. The event table compares pre-event demand against a preceding baseline and requires multiple adequately observed occurrences. It does not prove a festival caused demand growth. The actual dates are imported public Google Calendar entries, with provenance in Data Management. This is a snapshot, not continuous synchronization. Seasonal evidence sits inside Product Demand Forecasting; there is one combined forecast.

The review rule pipeline finds long catalog aliases first. A request for sugar-free cola maps to that SKU instead of ordinary cola. Availability, variant and quality intents can coexist. Evidence and original text remain visible. A no-history variant gets an investigation recommendation, not a fabricated forecast. The model never receives review counts.

Running a demonstration

1. Start the app using README commands and open the local link. State that all bundled records are synthetic.

2. Executive Dashboard: show actual recorded sales, draft quantities and the action queue.

3. Product Demand Forecasting: select Cola, switch7/14/30 days and explain the baseline comparison and dashed future line.

4. Smart Reorder Planner: inspect the formula explanation, then increase review days to observe constraints; restore default7 days.

5. Expiry Management: show yogurt risk, the expired coffee batch and the unknown chips expiry.

6. Slow-Moving Inventory: show biscuits overstock and why purchasing is paused.

7. Review Intelligence: show sugar-free requests; explain why no numeric order is invented.

8. Data Management: download templates, demonstrate an incomplete upload error and explain atomic replacement. Save/load is local and explicit.

Debugging guide

If the interface shows imports from the previous prototype, stop the old Streamlit process and launch a fresh one; Python may retain old modules across hot reloads. If a CSV fails, inspect the table/column named in the error and compare with its template. If forecast is unsupported, inspect recent observed counts and stockout masking. If an order is zero, inspect status, unknown expiry, horizon, cap and MOQ before blaming the model.

To reproduce predictions, run scripts/evaluate.py with single-thread BLAS/OpenMP. Check evaluation/metrics.json and the detailed prediction CSV. The test suite should pass before changing model behavior; after model changes rerun both tests and evaluation. Do not hand-edit metrics to make the report look better.

Boundaries of the demonstration

This is a local single-store proof of concept, with no purchase execution, public deployment, supported Revio/POS integration, authentication or verified ROI. Production requires consented data, real calendar maintenance, reconciliation, monitoring and human approvals. The previous unrelated review prototype is recoverable through Git, not mixed into the current implementation.

Part II - Engineering decisions

The initial sections below record v1 decisions and historical results. Revision2 sections supersede their model, calendar, evaluation and navigation choices; current metrics are in evaluation/metrics.json.

2026-10-17.

1. Data: generate 2024-01-01 through 2026-10-16 daily product history with seed 42. Include explicit daily opening/received/closing availability to prove sales conservation. Snapshot batches refer only to the as-of date. No claim of real retailer observations. Synthetic calendar is user-maintained and marked scenario-only, not an authoritative festival calendar.

2. Forecast: compare 28-day moving average, weekly seasonal naive and global HistGradientBoosting direct 1-30 day forecasts. Origin features use only prior uncensored observations; future calendar features are known, future sales never enter. Chronological validation origins choose a model; final 30-day test remains untouched until selection. Sparse/new products receive no exact order recommendation.

3. Inventory: periodic review with cumulative demand, explicit safety-days buffer (policy, not calibrated service guarantee), timed confirmed incoming orders and backorders. Simulate FEFO daily to estimate stock lost before use. Expired and unknown-expiry batches excluded from usable stock; unknown dates reported for verification. Cap order by shelf-life absorption and storage; MOQ/pack rounding cannot exceed cap.

4. Review intelligence: transparent product aliases and multi-label request/availability/quality intent rules. Review counts support investigation only and never multiply forecast units. No sentiment model retained.

5. UI/persistence: local Streamlit/Plotly with eight working sections; atomic six-table CSV upload; explicit SQLite save/load of validated datasets. Demo data runs offline. Uploaded records are not saved without clicking Save locally. No external posting or purchasing.

6. Evaluation: MAE, RMSE, WAPE, bias; chronological 30-day rollout and 7/14/30 prefix scores. Exclude censored targets from observed-sales accuracy and separately report simulation-only latent demand scores. Statistical validation does not establish real-world benefit.

Forecast implementation and measured selection

WHAT: Global direct HistGradientBoostingRegressor with 90 boosting iterations, 15 leaves, learning rate .08, L2=2 and random seed 42. WHY: nonlinear calendar/history effects without deep-network infrastructure. HOW: weekly historical origins after day 83 generate targets 1-30 days ahead; all demand inputs stop at origin and targets stop at each training cutoff. Product index is categorical; no review counts, latent demand, future stock or future sales enter features. Early stopping is disabled to avoid random internal time splits. ALTERNATIVES: moving average 28d and repeated last week. DECISION: pooled validation MAE 3.101 versus 3.639 and 4.435 selects boosting. EXAMPLE: last 30-day observed test MAE 2.975, WAPE 22.248%; this is simulation evidence. LIMITATIONS: pooled choice can hide weak SKUs; product identity does not generalize to unseen products; sparse/censored data biases learning; no calibrated intervals. Feature list is exported in source. Unavailable lags remain NaN, handled by the tree; baselines fill missing weekly values with past 28-day mean. Four model fits use expanding historical windows; final production fit uses all history.

Calendar and seasonal analysis

WHAT: known future event proximity plus cyclic month and weekday in the model, with descriptive per-product event comparisons. WHY: represent recurring changes without fixed universal summer multipliers. HOW: seven days before an event are compared with the preceding 28 days (same weekday proportions); at least two adequately observed events are required. ALTERNATIVES: manually uplift every SKU; rejected because unsupported. DECISION: calendar features are learned in regression, descriptive comparison never applies an extra multiplier. EXAMPLE: upcoming demo Diwali is a scenario, not an official date. LIMITATIONS: overlap, promotions and secular trends confound estimates; verified regional calendar required.

Reorder and expiry implementation

WHAT: periodic review with a daily FEFO simulation, timed confirmed receipts, pack/MOQ and receipt-life/storage caps. WHY: on-hand quantities may expire or arrive too late. HOW: usable = known future-expiry batches; compute target over lead + review days, add configurable safety-days times mean forecast, subtract eligible on-hand/incoming net of simulated losses and backorders. Stock expiring on a date may sell that day; next day it is unavailable. New-order quantity cannot exceed residual demand during its assumed receipt life or capacity. ALTERNATIVES: round every order up or use all on-hand regardless of expiry; rejected. DECISION: caps can deliberately leave target unmet; flag rather than fabricate feasible supply. EXAMPLE: 220 demand +30 safety -80 stock -20 incoming=150; 80 yogurt with five days at six/day leaves50 at risk. LIMITATIONS: deterministic forecasts; new supplier receipts assume catalog remaining life; no automatic purchasing; new-order caps are conservative, use stock at receipt before consumption, and require supplier confirmation. Unknown expiry blocks ordering; protection longer than30 days withholds quantities. Safety days are a policy buffer, not a calibrated service level.

Review, UI, persistence and testing

WHAT: transparent alias/English intent rules, eight Streamlit sections, atomic six-table snapshots and local SQLite. WHY: auditable prototype and easy local demonstration. HOW: longest nonoverlapping alias matches, explicit negation exceptions, no numerical review uplift; human review always required. ALTERNATIVES: LLM extraction would add cost, privacy and grounding work; multilingual embeddings deferred. DECISION: no paid keys required. EXAMPLE: sugar-free cola availability requests map to the new SKU but do not invent its sales. LIMITATIONS: sarcasm, multilingual wording, multi-product attribution and unfamiliar aliases may fail.

WHAT: numerical unit tests plus chronological evaluation and UI integration checks. WHY: recommendation correctness and prediction quality are different questions. HOW: known numeric examples, invalid schemas, missing/expired batches, timed POs, no-history suppression, leakage checks and end-to-end app tests.

ALTERNATIVES: screenshot-only testing misses math and validation failures. DECISION: tests cover business invariants; final browser inspection checks presentation. EXAMPLE: pack rounding cannot exceed safe cap. LIMITATIONS: passing synthetic tests does not validate a production retail operation.

Data design and feature construction details

WHAT: A six-table synthetic snapshot, availability ledger and origin-only feature matrix. WHY: reproducibility and explicit distinction between recorded sales, unknown demand and future targets. HOW: seed42 simulates latent demand, caps sales by stock and records receipts/writeoffs; validation checks the ledger and keys. Lags and rolling statistics use only the origin's history; missing/censored dates remain NaN. ALTERNATIVES: unbounded independent random sales would contradict stock, while filling missing days with zero would invent demand evidence. DECISION: retain latent demand solely for a separate simulation diagnostic and omit it from every feature. EXAMPLE: available3 and sold3 is conservatively masked, not labelled true demand3; a future sales change leaves earlier features identical. LIMITATIONS: masking is not a full censored-demand likelihood, and the synthetic generator can favor the chosen feature family.

Persistence and frontend details

WHAT: Streamlit session snapshots, content-keyed forecast caching and explicit SQLite save/load. WHY: a local assessment needs reproducible interactions without cloud infrastructure. HOW: all six tables validate before session replacement; SQLite writes a temporary database then replaces the saved file. UI upload controls and parser enforce a10MB limit. ALTERNATIVES: automatic database writes could persist accidental uploads; a hosted database would add unnecessary setup. DECISION: save only on an explicit button click and revalidate on load. EXAMPLE: an incomplete upload raises an error while the previous snapshot remains visible. LIMITATIONS: fixed local persistence is single-user; simultaneous writers, authentication and durable audit history are not addressed. Imported reviews update provenance even when numerical sales/evaluation remain unchanged.

Revision 2: unified seasonal forecasting and Google Calendar

WHAT: Added per-SKU Poisson regression with two annual Fourier harmonics, weekday indicators, linear time trend and individual festival preparation windows in one joint demand model. WHY: the original global model's month-level signals and one-month test did not establish robustness across seasons; a separate seasonal workspace obscured the single forecast. HOW: standardized known calendar features are fitted jointly with regularization alpha0.03; coefficients differ by SKU. No universal seasonal multiplier or second forecast is added. The former gradient-boosted model remains an unchanged comparator. ALTERNATIVES: manually increase summer demand, add a separate festival forecast, or tune models on the final test; rejected. DECISION: compare fixed candidates using earlier validation origins, freeze model choice, then report rolling-origin test errors across360 days, by season, SKU and event window. EXAMPLE: cola and tea can learn different annual curves while a gifting product learns a stronger festival effect. LIMITATIONS: harmonic regression assumes repeatable smooth seasonality and log-linear trend; abrupt changes and weather remain unmodelled. One year and180 observed days are required for annual fitting; otherwise use the28-day baseline and label the fallback.

WHAT: Imported public Holidays in India entries through the connected Google Calendar. WHY: replace fictional festival dates with the user's requested calendar source. HOW: keep only seven relevant festival families for2024-2027, preserve literal all-day dates and tentative labels, store public source links and retrieval date. Private, family and classroom events were not read. ALTERNATIVES: repeat fixed Gregorian dates or claim the connector automatically authenticates Streamlit; rejected. DECISION: an explicit imported snapshot, with calendar inspection/download/apply controls; no unattended live sync is claimed. EXAMPLE: Dussehra2026 is20October and Holi changes date each year. LIMITATIONS: calendar dates can change or

differ regionally; historical calendar retrieval is retrospective rather than a timestamped historical feed.

WHAT: Regenerated only the reproducible synthetic demo so simulated event demand aligns with imported calendar dates. WHY: relabelling fictional event-driven sales with real dates would corrupt evaluation. HOW: same seed and generator logic, public calendar inputs substituted; user uploads and local SQLite untouched. ALTERNATIVES: compare new errors directly with v1's different data; rejected. DECISION: all candidates are rerun on the identical v2 dataset; v1 metrics retained under evaluation/archive/v1. EXAMPLE: festival-related demand in2025 follows calendar Diwali20October rather than a fictional8November. LIMITATIONS: improvements remain simulation results and not evidence from a real retailer.

Revision3: product-pattern correction

The previous synthetic tea/coffee curve incorrectly peaked near October because it inverted the summer beverage curve. The generator now uses an explicit January-centered winter curve for hot drinks. This is a declared synthetic scenario correction, not evidence about a real merchant. Only bundled synthetic records were regenerated; user uploads and saved snapshots were untouched.

With only two or three occurrences per named festival, separate event coefficients overfit noise: tea received an unjustified Diwali dip. The model now pools festival preparation windows within each SKU while jointly fitting season, weekday and trend. This preserves a learned gifting uplift without forcing every product upward. Different named-festival effects cannot be estimated independently by this pooled model; richer history should support future hierarchical estimation.

Product checks: packaged sweets Diwali mean7.95 units/day versus4.20 on the same dates with event indicators omitted. This omitted-event value is a model diagnostic, not a second user-facing forecast or causal estimate. Tea and coffee seasonal expectations rise from October through December. Both remain within10% of their no-event values during Diwali in the synthetic no-hot-drink-festival-effect scenario. Tests cover both gifting SKUs, both hot drinks, Dussehra and Diwali. Shaded preparation windows and a summary of the same forecast clarify festival versus nonfestival days.

Full evaluation rerun: seasonal model test MAE2.860 versus same-data boosting2.964, n4,200 across360 days. These values cannot demonstrate improvement over v2 because the synthetic data changed. Repeated development against this simulation is not an untouched external benchmark. Prior metrics are retained in evaluation/archive/v2.

Part III - Interview preparation

Use these as explanations of this implementation, not claims of prior professional experience. Be clear about assistance used to build the project and demonstrate your understanding with the code and tests.

30-second pitch

StockSense is a proposed inventory intelligence module that could complement Revio. It helps a kirana owner forecast demand, review replenishment and identify expiry or excess stock. I compare simple baselines with seasonal Poisson regression and gradient boosting using chronological validation, then apply transparent inventory rules. Customer reviews suggest product investigations but never become invented sales quantities. The working demo uses synthetic data, so its results are engineering evidence, not proof of real-store savings.

Two-minute explanation

A retailer can have both stockouts and overstock. A sales forecast alone does not solve that because inventory may expire or arrive too late. StockSense validates sales, batches, orders, product rules, reviews and a calendar as one snapshot. It masks known stockout days rather than treating constrained sales as full demand.

One product-specific model combines season, weekdays, trend and festival dates into each daily forecast. Four validation windows select seasonal Poisson regression over boosting and two simple baselines. Twelve subsequent rolling windows test360 days. Synthetic observed MAE is2.860 versus2.964 for boosting and3.374 for the average.

The rule engine then simulates earliest-expiry-first consumption and dated receipts. It uses lead time plus review period, a configurable buffer, and conservative shelf-life, capacity, pack and MOQ limits to produce draft quantities. Unsupported products and unknown expiry require investigation. Reviews highlight requested variants or availability issues without numerically inflating forecasts. A production pilot would need consented real data, verified dates, integration and measured inventory outcomes.

Five-minute technical walkthrough

Minute1: Explain the merchant problem, Revio's advertised review-management context and why StockSense is a proposed complement, with feature overlap unverified.

Minute2: Open the six CSV templates. Explain availability-censored sales, recorded batch expiry, confirmed purchase orders and snapshot date. Show validation rejecting an incomplete upload without changing data.

Minute3: Show joint annual, weekday, trend and festival features in src/seasonal_model.py and the retained boosting comparator in src/forecasting.py. Explain why random splitting would leak future information. Display measured baseline comparisons and distinguish WAPE from accuracy.

Minute4: Work through220 demand +30 buffer -80 stock -20 incoming=150. Show FEFO and the80-unit yogurt/five-day example. Explain why pack rounding cannot exceed a safe cap and why an early shortage cannot be repaired by a late receipt.

Minute5: Show sugar-free review requests and the no-history warning. Explain uncalibrated uncertainty, synthetic data limits and the real-data shadow pilot needed before purchasing automation.

Questions and concise answers

1. What does the application do in plain language?

It helps a shop owner decide what stock needs attention: buy more, pause buying, inspect expiring batches or investigate a customer request. It explains each suggestion.

2. Does Revio already offer this?

Its public page advertises review management and related AI. I did not establish whether inventory exists privately. StockSense is a proposal; overlap must be checked with Cynbit.

3. Who would use this?

Kirana owners and purchasing staff are the proposed users. Revio publicly describes brand and location roles; kirana adoption is not verified.

4. Why choose this problem?

It links measurable ML forecasts to an operational decision and makes it possible to test business mathematics separately from model quality.

5. Where did the data come from?

A reproducible seed42 generator created the business records; public Google Calendar supplied actual festival dates. None are Cynbit data. The app also accepts a validated user-supplied snapshot.

6. Why not train directly on every sale?

Sales can be limited by stock. A shelf with three units cannot reveal whether demand was three or fifteen. Known constrained days are masked, although this does not fully solve censored-demand bias.

7. How do you handle missing dates?

They remain unknown after daily reindexing. They are not silently filled with zero demand.

8. Why seasonal Poisson regression?

It models nonnegative daily counts with smooth annual, weekday, trend and event effects learned jointly for each product. It beat boosting and simple baselines on pooled validation. Boosting remains a useful nonlinear comparator.

9. Why not LSTM or a large language model?

Neither is necessary for this small structured dataset. They add complexity, data needs or costs without demonstrated benefit here.

10. What are the main features?

The selected model uses two annual sine/cosine harmonics, weekday, trend and a pooled festival preparation window. The boosting comparator uses lags, rolling statistics, missingness, product identity, horizon, weekday, cyclic month and event proximity. No future stock, future sales or latent demand enters the predictors.

11. What does direct multi-horizon mean?

In the retained boosting comparator, each future day gets its own feature row from the same known origin history. Predictions are not fed back recursively as actual lags.

12. How is leakage prevented?

Features stop at the origin, labels stop at the training cutoff, validation follows training chronologically, and the final test does not choose the model. A causality test changes future sales and checks that earlier features stay identical.

13. What are the baselines?

A28-day observed average and a repeated last week. Missing weekly observations use only the prior28-day mean.

14. What happens if the baseline wins?

The selection rule chooses it. Model complexity is not the objective; validation performance is.

15. What are the measured results?

Seasonal Poisson rolling test MAE2.860, RMSE3.921, WAPE18.520%, bias-0.040. Same-data boosting MAE2.964; average MAE3.374. There are4,200 valid product-days across360 days, not real customer outcomes.

16. Is WAPE19% the same as81% accuracy?

No. WAPE is aggregate absolute numerical error divided by aggregate actual units. Subtracting it from100 does not produce classification accuracy.

17. Why report bias?

Persistent underprediction can cause shortages even when average absolute error looks acceptable. Positive bias means overprediction; negative means underprediction.

18. How are festivals handled?

Festival preparation windows are learned within the same forecast using actual public Google Calendar dates imported17 October2026. No second seasonal forecast or blanket uplift is added. The app uses a snapshot; regional/tentative dates need review.

19. Is the event uplift causal?

No. Trend, promotions and overlapping events can confound it. The evidence table does not apply an extra forecast multiplier.

20. How do you calculate an order?

Sum forecast over lead time plus review period, add safety buffer, subtract usable stock and confirmed incoming adjusted for expiry, add backorders, then apply capacity, receipt-life, MOQ and pack constraints.

21. Why use actual cumulative forecasts?

Demand can change by weekday or event. Summing daily forecasts preserves those changes instead of assuming the same daily average everywhere.

22. What is safety stock here?

A configurable number of demand-days. It is a transparent policy buffer, not a calibrated95% service-level calculation.

23. What if supply arrives too late?

The simulation flags shortages before arrival. It does not pretend later stock prevents earlier lost sales; the owner may need to expedite supply.

24. Why can the order be below the calculated target?

Shelf life, storage, pack sizes and MOQ can make the target infeasible. The system explains the binding constraint rather than ordering unsafe excess.

25. How does FEFO work?

Each day's demand consumes eligible stock with the earliest recorded expiry first. Expired and unknown-expiry batches do not supply the forecast.

26. Does AI predict the official expiry date?

No. Dates come from records. Forecasts estimate how many units might remain unsold before that date.

27. How is an expiry beyond30 days handled?

Its final unsold-at-expiry is unknown because demand is only forecast30 days. Blank risk is not zero risk.

28. How do reviews affect quantities?

They do not directly. Rules show product requests, availability, variants and quality signals for human investigation. There is no validated review-to-units conversion.

29. How do you avoid matching cola inside sugar-free cola?

Longer aliases claim their text spans first; overlapping shorter aliases are skipped.

30. What happens to a new product?

Without enough observed history it receives no precise demand forecast or purchase quantity. Requests support cautious investigation, not fabricated certainty.

31. How does CSV validation protect the workflow?

All six tables pass schema, dates, keys, numeric and reference checks before replacing the snapshot. A bad upload leaves the old state intact.

32. Why SQLite?

It provides simple optional local persistence without a separate service. It is not a substitute for tenant isolation or authenticated production storage.

33. What testing did you perform?

Numerical business-rule tests, invalid-data and persistence checks, feature causality, no-history/expiry/receipt/event/review cases and automated navigation of all seven UI sections. Evaluation is executed separately and saved.

34. What is the largest limitation?

External validity: synthetic data can only show that the approach works on the simulation. Real-store data quality, demand patterns and supplier behavior may differ substantially.

35. What would you do next?

Confirm Cynbit fit and interfaces, collect consented real data, run shadow recommendations, evaluate SKU-level errors and inventory costs, then add monitored authenticated deployment and human approval.

36. How would you measure business success?

Compare stockout hours, expiry cost, inventory holding and owner review time against a defined baseline while controlling for promotions, seasons and supply changes. Forecast MAE alone is insufficient.

37. What would you change if real demand is intermittent?

Evaluate intermittent-demand methods and probabilistic/count models against the same chronological protocol. Do not assume the current selected model will win.

38. Can this application place orders or be used publicly today?

No orders are placed. It runs locally without authentication. A public production service requires additional access controls, monitoring, consent and integration work.

39. Why is priority separate from confidence?

Urgency and certainty are different. An expired batch is urgent even if demand prediction is weak. A clear forecast does not make every product urgent.

40. What should you never overclaim in the interview?

Do not call synthetic data Cynbit data, claim a live Revio integration, assert a feature is absent without evidence, present WAPE as accuracy, or claim real-world savings from this simulation.